
Architectures Parallèles

TP2: x86_64 assembly - Floating-point arithmetic with SSE

UVSQ - M1 CHPS

0 Cheatsheet

0.1 Registers

Floating-point (FP) operations are typically performed using the XMM registers (`xmm0` to `xmm15`), which have been introduced by **SSE** (**S**treaming **S**IMD **E**xtension). The following figure lists the general purpose, special and XMM registers that we have seen so far:

Special registers

<code>rip</code>
<code>rflags</code>

General purpose registers

<code>rax*</code>
<code>rbx</code>
<code>rcx*</code>
<code>rdx*</code>
<code>rsi*</code>
<code>rdi*</code>
<code>rbp</code>
<code>rsp</code>
<code>r8*</code>
<code>r9*</code>
<code>r10*</code>
<code>r11*</code>
<code>r12</code>
<code>r13</code>
<code>r14</code>
<code>r15</code>

XMM registers

<code>xmm0*</code>
<code>xmm1*</code>
<code>xmm2*</code>
<code>xmm3*</code>
<code>xmm4*</code>
<code>xmm5*</code>
<code>xmm6*</code>
<code>xmm7*</code>
<code>xmm8*</code>
<code>xmm9*</code>
<code>xmm10*</code>
<code>xmm11*</code>
<code>xmm12*</code>
<code>xmm13*</code>
<code>xmm14*</code>
<code>xmm15*</code>

*: scratch register

Note that XMM registers are 128-bit wide. However, we will only use their full length later, after introducing SIMD instructions.

0.2 Instructions

Instruction	Description	C equivalent
<code>addsd x1, x2</code>	Double-precision FP addition	<code>x1 += x2;</code>
<code>addss x1, x2</code>	Single-precision FP addition	<code>x1 += x2;</code>
<code>subsd x1, x2</code>	FP subtraction	<code>x1 -= x2;</code>
<code>mulsd x1, x2</code>	FP multiplication	<code>x1 *= x2;</code>

<code>divs[d s] x1, x2</code>	FP addition	<code>x1 /= *x2;</code>
<code>sqrts[d s] x1, x2</code>	Square root	<code>x1 /= *x2;</code>
<code>movs[d s] x, m[64 32]</code>	Load value from memory to an XMM register	<code>x = *m32;</code>
<code>movs[d s] m[64 32], x</code>	Store value from an XMM register to memory	<code>*m64 = x;</code>
<code>cvtsi2s[d s] x, r[64 32]</code>	Cast a signed integer value to an FP value	<code>x = (double)r64;</code>
<code>cvtts[d s]2si r[64 32], x</code>	Cast an FP value to a signed integer value	<code>r32 = (int)x</code>

Similarly to the smaller versions of GPRs, scalar instructions that manipulate XMM registers target their **lower parts**.

Remark: As opposed to integer values, it is not possible to use immediate floating-point values as operands.

0.3 Using the XMM registers from inline assembly

In GCC inline assembly, XMM registers can be accessed using the "x" operand constraint.

Example in C:

```

1  double a = 41.;
2  asm(
3      "addsd %0, %1"
4      : "+x" (a)
5      : "x" (1.)
6      :
7  );

```

1 Simple 1D loops

1.1

```

1  void loop(size_t n, double a[n]) {
2      for(size_t i=0; i<n-1; ++i) {
3          a[i] = a[i+1] / sqrt(2.);
4      }
5  }

```

Write an equivalent function in ASM or inline ASM.

→ Notice the loop-carried dependence in this code. In the next lab, we will study how to exploit data-level parallelism on similar loops using **SIMD** (Single-Instruction, Mutiple-Data) instructions.

1.2

Which instruction can be moved out of the loop as an optimization? Using the code in the provided archive, observe the speedup yielded by this change.

2 Reciprocal square root

The goal of this exercise is to study and compare the different methods that have been used historically to optimize the computation of the reciprocal square root $\frac{1}{\sqrt{x}}$. This study will deal with both performance and numerical accuracy aspects. Nowadays, the most simple way to perform it is to use a hardware implementation in form of a dedicated instruction. However, doing so was not possible before such instructions were introduced (SSE was introduced in 1999, for instance).

In this exercise, you are given a set of scripts and drivers, which you are to complete with the different algorithm versions. Performance and accuracy measurements are handled by the driver. For more details, have a look at the attached README file.

Note: Before SSE was introduced, FP computations on x86 architectures were often performed using **x87** instructions. Even though the methods we are going to study have been made obsolete by SSE, we are still going to use it, as x87 is now mostly deprecated.

2.1 "Naive" approach

While x87 did not have an instruction for the reciprocal square root, it did have one for the square root. The "naive method" to compute the reciprocal square root would thus be to first compute the square root, and then its inverse:

```
1 float naive(float y) {
2     return 1. / sqrt(x);
3 }
```

Complete the naive function using inline ASM. Make sure to use the `sqrtps` and `divss` instructions. Launch the driver to observe the performance and accuracy results.

→ Both the square root and division instructions are known for having important latencies, which can make this method inefficient in certain contexts.

2.2 Newton-Raphson method

A very general method for approximating roots of real-valued functions is the Newton-Raphson method (or simply Newton method). It states that if x_0 is a satisfying initial guess for a root of $f(x)$ (that is $f(x_0) \approx 0$), then:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

is a better approximation. This method can be applied multiple times and will converge to the exact solution.

In the case of the reciprocal square root, we can write $y = \frac{1}{\sqrt{x}}$ such that $f(y) = \frac{1}{y^2} - x = 0$ is the function on which the Newton method should be applied. This yields, for all i , that $f'(y) = -\frac{2}{y^3}$ and thus:

$$y_{i+1} = \frac{y_i(3 - xy_i^2)}{2}$$

The following code computes an iteration of the Newton-Raphson method, given an x and an initial guess y :

```
1 float newton(float x, float y) {
2     const float threehalfs = 1.5f;
3     x2 = x * 0.5f;
4     return y * (threehalfs - (x2 * y * y));
5 }
```

Complete the newton function using inline ASM. You are free to add a second or third iteration of the Newton method. Launch the driver to observe the performance and accuracy results. It will use $y = 1$, which is a good approximation when $x \approx 1$. How does the accuracy evolve as x first approaches and then strays away from 1? How do performances compare with the naive approach?

2.3 Fast inverse square root

Before the introduction of hardware implementations, the **fast inverse square root** algorithm was the fastest way to compute reciprocal square roots. It also uses one or two iterations of the Newton-Raphson method, but is more known for its clever way of computing a first guess. Here is the C code for it:

```

1  float fast(float x) {
2      const float threehalfs = 1.5f;
3      float x2 = x * 0.5f;
4      float y = x;
5      long i = *(long *) &y;
6      i = 0x5f3759df - (i >> 1); // ???
7      y = *(float *) &i;
8      y = y * (threehalfs - (x2 * y * y));
9      // y = y * (threehalfs - (x2 * y * y));
10     return y;
11 }

```

Line 6 should be the most surprising one. The idea of this algorithm is to manipulate the floating-point input with bitwise operators, as if it was an integer (line 5). Without going into too much details, 0x5f3759df is a **magic number** which was chosen because it happened to give the best overall approximation when converting the value back to a `float`. One way of finding such constant is through a **trial-and-error process** (ie. testing many possible values and choosing the best one).¹

Complete the fast function using inline ASM. You are free to add a second or third iteration of the Newton method. Launch the driver to observe the performance and accuracy results. Conclude on the validity of the initial guess provided by this method.

¹For more details on the algorithm (especially the choice of the magic number), see this paper from Lomont: <http://www.lomont.org/papers/2003/InvSqrt.pdf>